

2026 SCSC 프로그래밍 경시대회 해설

Official Solutions

by

2026 SCPC 운영진

2026년 5월 16일

Staff

SCSC

운영

- 강명석 tomskang
- 김지훈 lycoris1600
- 정시우 sorohue

출제

- lycoris1600
- sjh1224
- sjhi00
- sorohue
- sungjae0506
- terrasphere
- ychangseok

검수

- flakepym
- hibye1217
- jthis
- mujigae
- qvixnh22
- realpsdoingdamyoo
- sadtreap
- utilforever
- yijw0930

Sponsor

SCSC



cenix820
gs12117
lycoris1600
queued_q
utilforever
집돌이 페렐만

cubic
Haru_101
martin0327
seokgukim
yijw0930

flakepym
Jank
plast
sungjae0506
Zye

코나미는피안신지원을내놔라

문제	의도한 난이도	출제자
3A 빠진 한 글자 찾기	Beginner	sjhi00
2A/3B Mobilint 텐서 스케줄링 (REGULUS)	Easy	lycoris1600
2F/3C 오름차순으로 정렬했을 때 K 번째 수	Easy	sjhi00
3D 스시스시 회전초밥	Easy	ychangseok
2J/3E DETOX	Easy	sorohue
3F SQL	Medium	sorohue
2G/3G SCSC 게임	Medium	terrasphere
3H 시험 공부	Medium	terrasphere
1A/2H/3I CUBRID HA Load Balance	Medium	lycoris1600
2D 색칠 놀이	Medium	terrasphere

문제		의도한 난이도	출제자
2C	재배열	Medium	sjh1224
1H/2B	SCSC 카드 놀이	Medium	terrasphere
1C	SCSC 마법의 화원	Medium	terrasphere
2E	마법의 상자 열기	Medium	terrasphere
2I	합과 차	Hard	terrasphere
1G	오fill	Hard	sorohue
1F	칩 연결하기	Hard	sorohue
1J/2L/3J	채널톡 워크플로우	Hard	sorohue

문제	의도한 난이도	출제자
2K 찬란한 연방의 놀이터에 대해	Hard	sorohue
1B 스시스시 유적	Hard	terrasphere
1I 트리와 게임과 퀴리	Hard	terrasphere
1K 롤케이크 보관	Hard	terrasphere
1E "저는 삼각기둥이 되고 싶어요"	Challenging	sungjae0506
1L 비전 마법사 루루	Challenging	terrasphere
1D Mobilint 텐서 스케줄링 (ARIES)	Challenging	lycoris1600

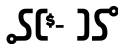
3A. 빠진 한글자 찾기

string, implementation

출제진 의도 – **Beginner**

- 처음 푼 참가자: **야!! 떤라!! 내가 캐리해줄께!! 뒷풀이로 이리하자!!!**, 0분
- 출제자: sjhi00

3A. 빠진 한 글자 찾기



- 풀이 1: 주어진 문자열이 CSC 또는 SCC이면 답은 S이고, SSC 또는 SCS이면 답은 C입니다. 이외의 문자열은 입력으로 주어지지 않으므로, 조건문으로 구현할 수 있습니다.
- 풀이 2: 문자열 SCSC에는 S가 2개, C가 2개 있습니다. 이중에서 문자 하나를 제거하면 S와 C 중 하나는 2개이고, 다른 하나는 1개가 됩니다. 주어진 문자열에서 문자 S와 C의 개수를 센 다음, S와 C 중에서 1개만 있는 문자를 출력하면 됩니다.
- 풀이 3: 모든 문자의 ASCII code 값을 bitwise XOR한 뒤, 그 값에 해당하는 문자를 출력해도 됩니다.

2A/3B. Mobilint 텐서 스케줄링 (REGULUS)

tree, ad-hoc

출제진 의도 - **Easy**

- **Div. 2** — 처음 푼 참가자: **RreShi**, 3분
- **Div. 3** — 처음 푼 참가자: **새봄**, 7분
- 출제자: lycoris1600

- 이 문제에서는 모든 텐서의 크기가 1 MiB입니다.
- 처음 메모리에 존재하는 텐서는 입력 텐서, 즉 리프 노드가 가리키는 텐서들입니다.
- 리프 노드의 개수를 L 이라고 하면 초기 메모리 사용량은 L MiB입니다.
- 리프가 아닌 노드 u 의 자식 수를 d 라고 합시다.
- u 번 텐서를 계산할 때는 먼저 크기 1의 텐서를 새로 할당하므로, 그 순간 메모리 사용량이 1 증가합니다.
- 계산이 끝나면 자식 텐서 d 개가 제거되므로, 계산 뒤 메모리 사용량 변화량은 $1 - d \leq 0$ 입니다.

- 따라서 어떤 계산을 한 번 끝낸 뒤의 메모리 사용량은 절대 증가하지 않습니다.
- 계산 중에는 새 텐서 하나를 할당하는 순간에만 메모리 사용량이 1 증가합니다.
- 그러므로 어떤 계산 순서를 택하더라도 피크 메모리 사용량은 $L + 1$ MiB를 넘지 않습니다.
- 반대로, $N \geq 2$ 이므로 리프가 아닌 노드가 적어도 하나 있습니다.
- 첫 번째로 결과 텐서를 계산하는 순간, 아직 메모리 사용량은 L MiB이고 새 텐서 하나를 할당해야 합니다.
- 따라서 피크 메모리 사용량은 반드시 $L + 1$ MiB 이상입니다.

- 결론적으로 정답은 항상 $L + 1$ 입니다.
- 제한에서 $N \leq 5000$, $M = 8192$ 이므로 항상 $L + 1 < M$ 입니다.
- 따라서 OOM인 경우는 없고, $L + 1$ 을 출력하면 됩니다.
- 리프 노드 수만 세면 되므로 전체 시간 복잡도는 $\mathcal{O}(N)$ 입니다.

2F/3C. 오름차순으로 정렬했을 때 K 번째 수

sorting

출제진 의도 - **Easy**

- **Div. 2** — 처음 푼 참가자: **운영진 판단에 따라 다른 이름**, 14분
- **Div. 3** — 처음 푼 참가자: **drk39**, 7분
- 출제자: **sjhi00**

- $i \geq N+1$ 인 모든 i 에 대하여 $a_i = a_{N+1}$ 임을 수학적 귀납법으로 증명할 수 있습니다.
- $i = N+1$ 인 경우는 자명합니다.
- 이제 $i = N+1, \dots, r$ ($r \geq N+1$) 일 때 성립한다고 가정하겠습니다.
- a_{N+1} 의 정의에 의하여, $a_1, \dots, a_N$ 중에서 a_{N+1} 보다 작은 수의 개수는 $K-1$ 개 이하이며, $a_1, \dots, a_N$ 중에서 a_{N+1} 보다 큰 수의 개수는 $N-K$ 개 이하입니다.
- 귀납 가정에 의하여, $a_{r-N+1}, \dots, a_r$ 중에서 a_{N+1} 보다 작은 수의 개수는 $K-1$ 개 이하이므로, $a_{r+1} \geq a_{N+1}$ 입니다. 또한, $a_{r-N+1}, \dots, a_r$ 중에서 a_{N+1} 보다 큰 수의 개수는 $N-K$ 개 이하이므로, $a_{r+1} \leq a_{N+1}$ 입니다. 즉, $a_{r+1} = a_{N+1}$ 이므로, $i = r+1$ 일 때도 성립합니다.

- 따라서 답은 매우 간단합니다. 만약 $M \leq N$ 이면 답은 a_M 입니다. 만약 $M > N$ 이면 답은 a_{N+1} 이며, $a_1, \dots, a_N$ 을 오름차순으로 정렬했을 때 K 번째에 해당하는 값입니다.
- 전체 시간복잡도는 $\mathcal{O}(N \log N)$ 입니다.

3D. 스시|스시 회전초밥

prefix_sum, greedy

출제진 의도 - **Easy**

- 처음 푼 참가자: **수달은 수학달콤**, 12분
- 출제자: ychangseok

- 먼저, 각 칸에서 먹을 수 있는 초밥의 최대 개수를 구해봅시다.
- i 번째 칸에서 먹을 수 있는 초밥의 최대 개수를 cnt_i 라고 하면, cnt_i 는 시뮬레이션을 통해서 구할 수 있습니다.
- 누적합을 이용하면 각 칸에서 위에서부터 초밥을 i 개 먹었을 때 얻는 만족감을 구할 수 있습니다.

- 따라서, 그리디 알고리즘을 이용하여 각 칸에서 얻을 수 있는 만족감의 최댓값을 구할 수 있습니다.
- 각 칸에서 얻을 수 있는 만족감의 최댓값을 모두 더하면 문제를 해결할 수 있습니다.
- 어떤 칸에서 초밥을 한 개도 먹지 않는 것이 최적인 경우와 $K_i = 0$ 인 경우를 주의해야 합니다.

2J/3E. DETOX

ad-hoc

출제진 의도 - Easy

- **Div. 2** — 처음 푼 참가자: **#pragma GCC target("furries")**, 16분
- **Div. 3** — 처음 푼 참가자: **opal2718**, 38분
- 출제자: sorohue

- 자신의 명찰을 ?로 표기합니다.
- 00?, 0?0, ?00와 같이 자신의 명찰에 따라 연속한 세 학생이 같은 명찰을 달고 있게 될 수 있으면 그 학생은 첫 라운드에 자신의 명찰에 적힌 문자를 확정할 수 있습니다.
- 첫 라운드에 자신의 명찰에 적힌 문자를 바로 확신할 수 없는 경우는 0X?0X 꼴 뿐입니다.

- 명찰을 길이 1짜리 블록의 체인(0X0X0X..., 이하 1-체인)과 길이 2짜리 블록의 체인(00XX00XX..., 이하 2-체인)으로 분할합니다.
- 0X?0X는 자신의 명찰에 따라 0X00X 또는 0XX0X 꼴일 수 있습니다. 두 경우 모두 **자신의 명찰이 2-체인의 한쪽 끝임을** 관찰합니다.
- 나아가 모든 2-체인의 끝이 0X?0X 꼴로 나타난다는 사실 역시 관찰할 수 있습니다.

- 따라서 첫 번째 라운드에 거수하지 않은 학생들이 곧 모든 2-체인의 양쪽 끝과 같습니다. 2-체인의 양쪽 끝 학생이 **모두** 거수하지 않으므로, 거수하지 않은 학생은 두 번째 라운드에 적어도 한 명의 다른 거수하지 않은 학생을 확인할 수 있습니다.
- 이때 그 학생이 달고 있는 명찰과 같은 문자가 적힌 명찰을 달고 있는 학생이 어느 쪽에 앉아 있는지를 확인하면 2-체인과 1-체인의 순서를 반드시 알 수 있습니다. 이로부터 자신에게 연결된 2-체인이 왼쪽과 오른쪽 중 어느쪽인지 확신할 수 있습니다.
- 따라서 첫 번째 라운드에 거수하지 않은 학생들은 **모두** 두 번째 라운드에 거수합니다.

3F. SQL

graphs

출제진 의도 – **Medium**

- 처음 푼 참가자: **drk39**, 65분
- 출제자: sorohue

- 각 쿼리를 정점으로 보고 출력을 출력하는 쿼리마다 해당 쿼리에서 나가는 하나의 단방향 **참조 간선**을 만들어 그래프를 구성합니다.
- 참조 간선을 따라가다 보면 반드시 참조 간선의 사이클 또는 입력을 출력하는 쿼리 중 하나에 도달하게 됩니다.
- 간선으로 연결된 쿼리는 모두 출력이 같아야 합니다.
 - 입력을 출력하는 쿼리가 있는 경우 해당 입력으로 통일합니다.
 - 참조 간선의 사이클이 있는 경우(입력을 출력하는 쿼리가 없는 경우) 범위 내의 아무 값으로나 통일해도 괜찮습니다.

2G/3G. SCSC 게임

game_theory, parity, ad_hoc

출제진 의도 - **Medium**

- **Div. 2** — 처음 푼 참가자: **Jank**, 34분
- **Div. 3** — 처음 푼 참가자: **aboisabo**, 86분
- 출제자: terrasphere

- 현재 문자열에는 SCSC가 없습니다.
- 문자를 하나 지운 뒤 SCSC가 새로 생기려면, 지운 위치 양쪽이 이어지면서 SCSC가 되어야 합니다.
- 즉, 지우기 전에는 SCSC에 문자 하나가 끼어 있는 꼴이어야 합니다.
- 가능한 경우는 SCCSC, SCSSC 두 가지뿐입니다.
- 따라서 현재 문자열에 둘 중 하나가 있으면, 현재 플레이어가 즉시 승리할 수 있습니다.

- 이제 SCCSC, SCSSC가 모두 없는 상태를 **안전한 상태**라고 부릅니다.
- 안전한 상태에서는 이번 턴에 바로 이길 수 없습니다.
- 하지만 맨 뒤 문자를 지우면 새로운 인접 관계가 생기지 않습니다.
- 따라서 안전한 상태에서는 항상 상대에게 안전한 상태를 넘기는 수가 존재합니다.

- 안전한 상태에서는 서로 즉시 승리를 주지 않는 방향으로 플레이할 수 있습니다.
- 그러면 게임은 단순히 문자열 길이를 1 씩 줄이는 게임처럼 됩니다.
- 빈 문자열은 현재 플레이어가 질 수밖에 없으므로 패배 상태입니다.
- 따라서 안전한 상태에서는 길이가 짝수이면 패배, 홀수이면 승리입니다.

- 따라서 Terra가 이기는 경우는 다음 두 가지입니다.
 - 문자열 길이가 홀수인 경우
 - 문자열 길이가 짝수이고 SCCSC 또는 SCSSC가 있는 경우
- 나머지 경우에는 Lulu가 이깁니다.
- 전체 시간 복잡도는 $\mathcal{O}(\sum |S_i|)$ 입니다.

3H. 시험 공부

tree

출제진 의도 - **Medium**

- 처음 푼 참가자: **새봄**, 127분
- 출제자: terrasphere

- 어떤 순서로 K 개의 주제를 공부한다고 합시다.
- 한 주제를 공부할 때 이미 공부한 인접 주제 하나마다 B 만큼 시간이 줄어듭니다.
- 이 시간 단축은 결국, 선택된 두 주제를 잇는 간선 하나에서 생깁니다.
- 트리에서 K 개의 정점 사이의 간선 수는 최대 $K - 1$ 개입니다.

3H. 시험 공부

- 따라서 전체 과정에서 시간 단축에 사용될 수 있는 간선은 최대 $K - 1$ 개입니다.
- $A > B$ 라면, K 개의 주제를 공부하는 데 걸리는 시간은 적어도

$$KA - B(K - 1) = A + (K - 1)(A - B)$$

입니다.

- $A = B$ 라면 첫 주제는 A 시간, 이후 주제들은 각각 적어도 1 시간이 필요합니다.
- 두 경우를 합치면 최소 시간의 하한은

$$A + (K - 1) \max(1, A - B)$$

입니다.

- 이 하한은 실제로 달성 가능합니다.
- 아무 정점을 루트로 잡고 BFS 또는 DFS 순서로 정점들을 나열합니다.
- 루트가 아닌 모든 정점은 자신의 부모가 먼저 등장합니다.
- 따라서 첫 정점은 A 시간, 이후 각 정점은

$$\max(1, A - B)$$

시간에 공부할 수 있습니다.

- $C = \max(1, A - B)$ 라고 합시다.
- $T < A$ 라면 아무 주제도 공부할 수 없습니다.
- 그렇지 않다면 첫 주제를 공부한 뒤 남은 시간은 $T - A$ 입니다.
- 따라서 공부할 수 있는 최대 주제 수는

$$K = 1 + \min\left(N - 1, \left\lfloor \frac{T - A}{C} \right\rfloor\right)$$

입니다.

- 이때 K 개의 주제를 공부하는 최소 시간은

$$M = A + (K - 1)C$$

입니다.

- 실제 순서는 BFS 또는 DFS 순서의 앞 K 개를 출력하면 됩니다.
- 전체 시간 복잡도는 $\mathcal{O}(N)$ 입니다.

1A/2H/3I. CUBRID HA Load Balance

greedy

출제진 의도 - **Medium**

- **Div. 1** — 처음 푼 참가자: **ibm2005**, 12분
- **Div. 2** — 처음 푼 참가자: **chunbae**보다못하는사람, 177분
- **Div. 3** — 처음 푼 참가자: **jAnk**는 실존한다, 179분
- 출제자: lycoris1600

- 현재 살아 있는 초기 Master 서버 수를 x , 초기 Slave 서버 수를 y 라고 합시다.
- $x > 0$ 이면 실제 Master/Slave 서버 수는 그대로 x, y 입니다.
 - 따라서 쿼리 2 a b c를 처리할 조건은

$$x \geq a, \quad y \geq c, \quad x + y \geq a + b + c$$

입니다.

- $x = 0$ 이고 $y > 0$ 이면 Slave 하나가 즉시 Master가 되므로, 실제 Master/Slave 서버 수는 $1, y - 1$ 입니다.
- $x = y = 0$ 이면 요청이 하나도 없는 경우에만 처리할 수 있습니다.

1A/2H/3I. CUBRID HA Load Balance

- 쿼리를 거꾸로 봅시다.
- 정방향에서 서버를 끄는 쿼리 1 i 는, 역방향에서는 i 번 서버를 새로 사용할 수 있게 되는 사건입니다.
 - 같은 서버를 여러 번 끄는 경우, 실제로 의미 있는 것은 정방향에서의 첫 번째 1 i 뿐입니다.
- 역방향으로 보면서, 지금 시점에 살아 있는 것으로 고른 서버 수를 k 라고 합시다.
- 이 k 개 중 초기 Master 서버 수로 가능한 값들의 구간을 $[L, R]$ 로 관리합니다.
- 새 서버 하나를 추가로 고르면, 그 서버를 Slave로 둘 수도 있고 Master로 둘 수도 있으므로

$$k \leftarrow k + 1, \quad [L, R] \leftarrow [L, R + 1]$$

로 바뀝니다.

- 쿼리 2 a b c를 현재 선택한 k 개의 서버로 처리할 수 있는 초기 Master 수의 구간을 구합시다.
- $T = a + b + c$ 라고 하겠습니다.
- $T = 0$ 이면 가능한 구간은 $[0, k]$ 입니다.
- $T > 0$ 이고 $k < T$ 이면 어떤 역할 배정으로도 처리할 수 없습니다.
- 이제 $T > 0$ 이고 $k \geq T$ 라고 합시다.
 - 초기 Master가 없는 경우는 $a \leq 1$ 이고 $k \geq c + 1$ 일 때 가능하며, 이때 Master 수는 0입니다.
 - 초기 Master가 있는 경우는 Master 수 x 가 $\max(1, a) \leq x \leq k - c$ 를 만족해야 합니다.
- 두 경우를 합치면 가능한 Master 수는 하나의 구간으로 표현됩니다.

- 처음에는 역방향 맨 끝에서 시작합니다.
 - 한 번도 꺼지지 않는 서버만 사용할 수 있는 상태입니다.
 - 아직 고른 서버는 없으므로 $k = 0, [L, R] = [0, 0]$ 입니다.
- 역방향으로 쿼리를 보면서 다음을 수행합니다.
 1. 1 i 가 i 번 서버의 정방향 첫 번째 종료 쿼리라면, 사용할 수 있는 서버 수를 1 늘립니다.
 2. 2 a b c를 만나면, 현재 k 에 대해 가능한 Master 수 구간 I 를 계산합니다.
 3. $[L, R] \cap I$ 가 비어 있는 동안, 사용할 수 있는 서버를 하나 더 골라 k 와 R 을 1씩 늘립니다.
 4. 더 고를 수 있는 서버가 없는데도 교집합이 비어 있으면 답은 -1 입니다.
 5. 교집합이 생기면 $[L, R] \leftarrow [L, R] \cap I$ 로 갱신합니다.

- 이 그리디가 맞는 이유는 다음과 같습니다.
- 역방향에서 서버를 추가로 고르는 것은 이미 처리한 더 뒤쪽 쿼리들에 대해 서버를 더 많이 켜 두는 것과 같습니다.
- 서버가 더 많아지면 처리 가능성이 나빠지지 않으며, 가능한 초기 Master 수의 구간은 오른쪽으로만 넓어집니다.
- 따라서 현재 쿼리를 처리할 수 없다면, 어떤 올바른 해도 이 시점에 살아 있는 서버를 더 골라야 합니다.
- 그러므로 필요해지는 순간마다 서버를 하나씩 추가하는 그리디가 최소 서버 수를 만듭니다.
- 각 서버는 최대 한 번만 추가되므로 전체 시간 복잡도는 $\mathcal{O}(N + Q)$ 입니다.

2D. 색칠놀이

greedy, prefix_sum

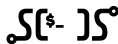
출제진 의도 - **Medium**

- 처음 푼 참가자: **상등병의편지**, 56분
- 출제자: terrasphere

- 구간 안의 모든 칸을 세 색 중 하나로 칠해야 합니다.
- X인 칸은 아직 비어 있으므로, 원하는 색으로 칠해도 덧칠 횟수에 포함되지 않습니다.
- 이미 칠해진 칸도 다른 색으로 덧칠할 수 있지만, 이때는 비용이 1 듭니다.
- 목표는 인접한 두 칸의 색이 모두 다르도록 만드는 것입니다.

- 서로 다른 색이 인접한 부분은 이미 조건을 만족합니다.
- 사용할 수 있는 색이 3가지이기 때문에, X가 있는 부분은 언제나 원하는 색을 골라서 맞출 수 있습니다.
- 따라서 문제가 되는 것은 이미 같은 색으로 칠해진 연속 구간뿐입니다.
- 예를 들어 RRRRRR 같은 구간은 그대로 둘 수 없습니다.

2D. 색칠 놀이



- 길이가 k 인 같은 색 연속 구간을 생각합니다.

$$\underbrace{RR \cdots R}_k$$

- 인접한 같은 색 쌍을 없애려면, 최소한 한 칸 건너 하나씩은 덧칠해야 합니다.
- 따라서 필요한 덧칠 횟수는 적어도

$$\left\lceil \frac{k}{2} \right\rceil$$

입니다.

- 실제로 짝수 번째 칸들을 다른 색으로 칠하면 이 하한을 달성할 수 있습니다.

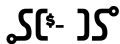
- 따라서 같은 색 연속 구간 하나의 비용은

$$\left\lfloor \frac{\text{길이}}{2} \right\rfloor$$

입니다.

- 결국 각 쿼리는 구간 안에 포함된 연속 구간들의 비용 합을 묻는 문제가 됩니다.

2D. 색칠 놀이



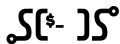
- 문자열을 같은 문자끼리의 최대 연속 구간으로 나눕니다.
- 각 block에 대해 시작 위치, 끝 위치, 문자, 비용을 저장합니다.
- 문자가 X이면 비용은 0입니다.
- 문자가 R, G, B이면 비용은

$$\left\lfloor \frac{\text{block length}}{2} \right\rfloor$$

입니다.

- 각 위치가 어느 block에 속하는지도 저장해 둡니다.
- 또한 block 비용의 prefix sum을 만들어 둡니다.
- 그러면 쿼리 구간 $[l, r]$ 안에 완전히 포함된 block들의 비용은 prefix sum으로 바로 구할 수 있습니다.

2D. 색칠 놀이



- 쿼리 $[l, r]$ 을 생각합니다.
- l 이 속한 block 을 L , r 이 속한 block 을 R 이라고 합니다.
- 만약 $L = R$ 이라면, 한 연속 구간의 일부만 잘라낸 것입니다.
- 이 경우 답은

$$\begin{cases} 0 & (\text{문자가 } x) \\ \left\lfloor \frac{r-l+1}{2} \right\rfloor & (\text{그 외}) \end{cases}$$

입니다.

- 이제 $L < R$ 인 경우입니다.
- 왼쪽 끝 block에서는 l 부터 block의 끝까지의 길이만 봅니다.
- 오른쪽 끝 block에서는 block의 시작부터 r 까지의 길이만 봅니다.
- 가운데 완전히 포함된 block들은 prefix sum으로 더합니다.

2D. 색칠 놀이

- 끝 block의 문자가 X이면 그 부분의 비용은 0입니다.
- 그렇지 않다면 길이가 k 인 잘린 부분의 비용은

$$\left\lfloor \frac{k}{2} \right\rfloor$$

입니다.

- 따라서 각 쿼리는 block 번호만 알면 $\mathcal{O}(1)$ 에 처리됩니다.
- 전체 시간 복잡도는 $\mathcal{O}(N + Q)$ 입니다.

2C. 재배열

Z, greedy

출제진 의도 – Medium

- 처음 푼 참가자: **상등병의편지**, 20분
- 출제자: sjh1224

- 배열 B 를 최대한 많이 등장시키려면, 여러 개의 B 가 서로 최대한 많이 겹치도록 배치하는 것이 유리함을 증명할 수 있습니다.
- 예를 들어 B 의 접두사와 접미사가 길이 x 만큼 같다면, 두 B 를 $M - x$ 칸 차이로 시작하게 하여 x 칸을 겹칠 수 있습니다.
- 따라서 B 의 접두사이면서 접미사인 가장 긴 부분 배열의 길이를 구하면 됩니다.
- 이는 KMP의 prefix function이나 Z algorithm으로 $O(M)$ 에 구할 수 있습니다.

2C. 재배열

- B 의 접두사이면서 접미사인 가장 긴 부분 배열의 길이를 x 라고 합시다.
- 그러면 B 하나를 놓은 뒤, 다음 B 를 만들기 위해 새로 추가해야 하는 부분은

$$B_{x+1}, B_{x+2}, \dots, B_M$$

뿐입니다.

- 이 배열을 C 라고 합시다.
- 즉, 답은 다음과 같은 꼴로 만들 수 있습니다.

$$B + C + C + C + \dots$$

- 이때 C 를 하나 더 붙일 때마다 B 의 등장 횟수가 1씩 증가합니다.

- 이제 A 는 순서를 마음대로 바꿀 수 있으므로, 중요한 것은 각 값이 몇 개 있는지입니다.
- 먼저 B 를 한 번 만들 수 있는지 확인합니다.
 - 만들 수 없다면 B 는 한 번도 등장할 수 없으므로, A 의 원소를 아무 순서로 출력하면 됩니다.
- 만들 수 있다면, A 에서 B 의 원소들을 제거하고 답의 앞에 B 를 출력합니다.
- 그 뒤에는 C 를 만들 수 있는 동안 계속 C 를 붙입니다.
- 더 이상 C 를 만들 수 없으면, 남은 원소들은 답의 뒤에 아무 순서로 붙이면 됩니다.

- 구현 방법은 다음과 같습니다.
 1. KMP 또는 Z algorithm으로 x 를 구합니다.
 2. 각 값의 개수를 세어 둡니다.
 3. B 를 한 번 만들 수 있으면 답에 B 를 추가하고, 개수를 차감합니다.
 4. $C = [B_{x+1}, \dots, B_M]$ 를 만들 수 있는 동안 답에 C 를 추가하고, 개수를 차감합니다.
 5. 남은 원소들을 아무 순서로 출력합니다.
- 시간 복잡도는 counter 배열을 사용하면 $O(N + M + \max A_i)$, 또는 `std::map`을 사용하면 $O((N + M) \log N)$ 입니다.

1H/2B. SCSC 카드 놀이

greedy

출제진 의도 - Medium

- **Div. 1** — 처음 푼 참가자: **XX**, X분
- **Div. 2** — 처음 푼 참가자: **XX**, X분
- 출제자: terrasphere

- 현재 줄에 남아 있는 한 사람은, 사실 어떤 연속 구간을 대표합니다.
- 그 사람이 가진 카드들 중 중요한 값은 두 개뿐입니다.

$(s, c) = (\text{가장 작은 스페이드}, \text{가장 작은 클로버})$

- 두 구간을 합칠 때도 (s, c) 만 알면 결과를 알 수 있습니다.

- 두 상태가 $(s_1, c_1), (s_2, c_2)$ 라고 합시다.
- S-게임에서는 스페이드 최솟값이 더 큰 쪽이 이깁니다.

$$(s_1, c_1), (s_2, c_2) \rightarrow (\max(s_1, s_2), \min(c_1, c_2))$$

- C-게임에서는 클로버 최솟값이 더 큰 쪽이 이깁니다.

$$(s_1, c_1), (s_2, c_2) \rightarrow (\min(s_1, s_2), \max(c_1, c_2))$$

- 최종적으로 얻을 값을 (S_f, C_f) 라고 합시다.
- 목적 함수는

$$(S_{\text{sum}} - S_f)(C_{\text{sum}} - C_f)$$

입니다.

- S_f 를 고정하면, 이 값은 C_f 가 클수록 작아집니다.
- 따라서 각 S_f 에 대해 가능한 최대 C_f 만 구하면 됩니다.

- 최종 스페이드 값을 p 로 고정합니다.
- $S_i \geq p$ 인 부원을 **큰 부원**, $S_i < p$ 인 부원을 **작은 부원**이라고 부르겠습니다.
- 최종 스페이드 최솟값이 p 가 되려면, 작은 부원의 스페이드 카드는 마지막까지 남으면 안 됩니다.
- 따라서 작은 부원들은 결국 S-게임에서 패배해 사라져야 합니다.

- 작은 부원들로만 이루어진 연속 구간을 생각합시다.
- 이 구간은 C-게임만 사용해 하나로 합칠 수 있습니다.
- 그러면 그 구간의 상태는

$$(\min S_i, \max C_i)$$

끝이 됩니다.

- 이후 인접한 큰 부원과 S-게임을 하면, 작은 스페이드는 사라지고 클로버 값만 큰 부원 쪽으로 넘어갑니다.

- $S_i \geq p$ 인 부원들의 가장 왼쪽 위치를 L , 가장 오른쪽 위치를 R 이라고 합시다.
- $[L, R]$ 바깥쪽에는 작은 부원만 있습니다.
- 왼쪽 바깥 구간의 최대 클로버를 A_L , 오른쪽 바깥 구간의 최대 클로버를 A_R 이라고 둡니다.
- 구간이 비어 있으면 해당 값은 ∞ 로 생각합니다.

- 먼저 $L < R$ 인 경우를 봅시다.
- L 과 R 사이에 있는 큰 부원은 바깥 작은 구간을 직접 처리할 필요가 없습니다.
- 따라서 내부 큰 부원은 자신의 클로버 값 C_i 를 그대로 최종 C_f 후보로 만들 수 있습니다.

$$\max_{L < i < R, S_i \geq p} C_i$$

- 이제 양 끝 큰 부원 L, R 을 봅시다.
- L 이 최종 클로버를 담당하려면, 왼쪽 바깥의 작은 부원들을 처리해야 합니다.
- 왼쪽 작은 구간은 최대 클로버 A_L 을 가진 하나의 상태로 합칠 수 있으므로, L 쪽에서 남길 수 있는 클로버 값은

$$\min(C_L, A_L)$$

입니다.

- 오른쪽 끝 R 도 마찬가지로 $\min(C_R, A_R)$ 를 후보로 줍니다.

- 따라서 $L < R$ 일 때 가능한 최대 C_f 는

$$\max \left(\max_{L < i < R, S_i \geq p} C_i, \min(C_L, A_L), \min(C_R, A_R) \right)$$

입니다.

- 내부의 작은 구간들은 적당한 인접 큰 부원에게 S-게임으로 제거하면 됩니다.
- 큰 부원들끼리는 C-게임으로 합치면, 클로버 값은 최대값으로 유지됩니다.

- 이제 $L = R$ 인 경우를 봅시다.
- 이때 $S_i \geq p$ 인 부원은 p 를 가진 부원 하나뿐입니다.
- 이 부원이 왼쪽과 오른쪽의 작은 구간을 모두 처리해야 합니다.
- 따라서 가능한 최대 클로버 값은

$$\min(C_L, A_L, A_R)$$

입니다.

- 이제 모든 부원 i 에 대해 $p = s_i$ 를 최종 스페이드로 고정합니다.
- 앞의 방법으로 가능한 최대 C_f 를 계산합니다.
- 그때의 후보 정답은

$$(S_{\text{sum}} - p)(C_{\text{sum}} - C_f)$$

입니다.

- 이 값의 최솟값을 출력합니다.
- 전체 시간 복잡도는 $\mathcal{O}(N^2)$ 입니다.

1C. SCSC 마법의 화원

constructive, ad_hoc

출제진 의도 – Medium

- 처음 푼 참가자: **ultradiaxon-n3**, 31분
- 출제자: terrasphere

1C. SCSC 마법의 화원

- 모든 3×3 부분격자에서 S가 C보다 많아야 합니다.
- 즉, 모든 3×3 부분격자에는 S가 적어도 5개 있어야 합니다.
- 반대로 전체 격자에서는 C가 더 많아야 하므로,

$$\#S < \frac{N^2}{2}$$

이어야 합니다.

- 이제 가능한 S 개수의 하한을 구해 봅시다.

1C. SCSC 마법의 화원

- 먼저 $N = 3q$ 인 경우를 봅시다.
- 격자를 q^2 개의 3×3 블록으로 완전히 타일링할 수 있습니다.
- 각 블록에는 S가 적어도 5개 필요하므로,

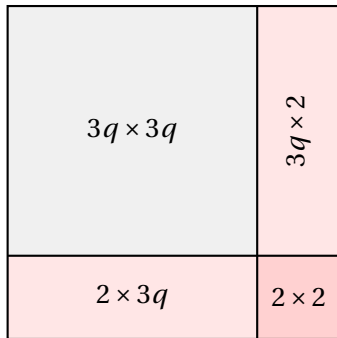
$$\#S \geq 5q^2 = \frac{5}{9}N^2$$

입니다.

- 이는 $\frac{N^2}{2}$ 보다 크므로, 전체에서 C가 더 많을 수 없습니다.
- 따라서 $3 \nmid N$ 이면 불가능합니다.

1C. SCSC 마법의 화원

SCSC



$N = 3q + 2$ 인 경우의 분해

- 왼쪽 위의 $3q \times 3q$ 부분에는 S가 적어도 $5q^2$ 개 필요합니다.
- 오른쪽 $3q \times 2$ strip을 q 개의 3×2 조각으로 나눕니다.
- 어떤 3×2 조각에 S가 1개 이하라면, 옆의 한 열을 붙인 3×3 안에는 S가 많아야 4개입니다.
- 따라서 오른쪽 strip에는 S가 적어도 $2q$ 개 필요합니다.
- 아래쪽 $2 \times 3q$ strip도 같은 이유로 적어도 $2q$ 개가 필요합니다.

1C. SCSC 마법의 화원

- 따라서 $N = 3q + 2$ 일 때

$$\#S \geq 5q^2 + 4q$$

입니다.

- 전체에서 C가 더 많으려면

$$5q^2 + 4q < \frac{(3q + 2)^2}{2}$$

이어야 합니다.

- 이를 정리하면

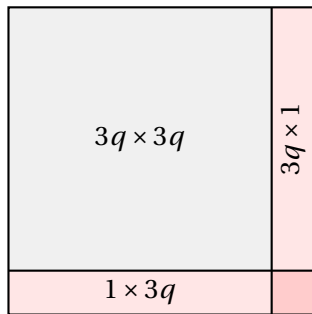
$$q^2 - 4q - 4 < 0$$

이고, 정수 $q \geq 1$ 에 대해 $q \leq 4$ 에서만 성립합니다.

- 따라서 $N = 3q + 2$ 에서는 $N \geq 17$ 이면 불가능합니다.

1C. SCSC 마법의 화원

SCSC



$N = 3q + 1$ 인 경우의 분해

- 왼쪽 위의 $3q \times 3q$ 부분에는 S가 적어도 $5q^2$ 개 필요합니다.
- 이제 마지막 행과 마지막 열로 이루어진 오른쪽 아래 사이드를 봅시다.
- 이 사이드가 전부 C일 수는 없습니다.

- 만약 사이드가 전부 C라면, 가장 오른쪽 아래에 위치한 3×3 부분격자에서 마지막 행과 마지막 열에 해당하는 5칸이 모두 C가 됩니다.
- 이는 조건인 "모든 3×3 에서 S가 적어도 5개"와 모순입니다.
- 따라서 사이드에는 S가 적어도 1개 필요합니다.

1C. SCSC 마법의 화원

- 따라서 $N = 3q + 1$ 일 때

$$\#S \geq 5q^2 + 1$$

입니다.

- 전체에서 C가 더 많으려면

$$5q^2 + 1 < \frac{(3q + 1)^2}{2}$$

이어야 합니다.

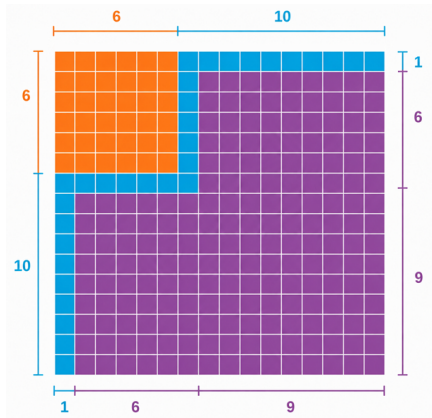
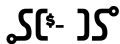
- 이를 정리하면

$$q^2 - 6q + 1 < 0$$

이고, 정수 $q \geq 1$ 에 대해 $q \leq 5$ 에서만 성립합니다.

- 따라서 $N = 3q + 1$ 에서는 $N \geq 19$ 이면 불가능합니다.

1C. SCSC 마법의 화원



- 그림처럼 사이드를 꺾어서 보면, 나머지 네 직사각형은 모두 3×3 블록으로 타일링할 수 있습니다.
- 네 직사각형의 전체 넓이는 15×15 이므로, 이 부분에만 S가 적어도 $25 \cdot 5 = 125$ 개 필요합니다.

- 이제 그림의 꺾이는 코너들을 봅시다.
- 코너 하나는 어떤 3×3 부분격자의 5칸을 차지합니다.
- 이 5칸이 전부 C라면, 나머지 4칸이 모두 S여도 조건을 만족할 수 없습니다.
- 따라서 각 코너에는 S가 적어도 1개 필요합니다.
- 그림에서 서로 겹치지 않는 코너가 3개 있으므로, 사이드에서만 S가 적어도 3개 더 필요합니다.

1C. SCSC 마법의 회원

- 따라서 16×16 격자 전체에서 필요한 S의 개수는 적어도

$$125 + 3 = 128$$

개입니다.

- 하지만 전체 칸 수는 256개이므로, C가 더 많으려면 S는 많아야 127개여야 합니다.
- 모순이므로 $N = 16$ 도 불가능합니다.
- 따라서 이제 남은 가능성은

$$N < 16, \quad 3 \nmid N$$

뿐입니다.

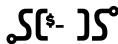
- 남은 경우에는 실제로 회원을 구성할 수 있습니다.
- $N < 16$ 이고 $3 \nmid N$ 이므로,

$$N \in \{4, 5, 7, 8, 10, 11, 13, 14\}$$

입니다.

- $N \bmod 3$ 에 따라 두 가지 패턴을 사용합니다.

1C. SCSC 마법의 화원

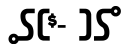


- 먼저 $N \equiv 2 \pmod{3}$ 인 경우입니다.
- 다음 3×3 패턴을 주기적으로 반복합니다.

C	C	S
C	C	S
S	S	S

- 즉, 행 번호나 열 번호가 3의 배수이면 S, 아니면 C를 둡니다.
- 모든 3×3 부분격자에는 S가 정확히 5개 들어갑니다.

1C. SCSC 마법의 화원



- $N = 3q + 2$ 일 때 이 패턴의 S 개수는

$$5q^2 + 4q$$

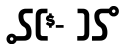
입니다.

- 현재 가능한 범위에서는 $q \leq 4$ 입니다.
- 앞에서 본 부등식에 의해

$$5q^2 + 4q < \frac{(3q + 2)^2}{2}$$

이므로 전체에서는 C가 더 많습니다.

1C. SCSC 마법의 화원



- 다음으로 $N \equiv 1 \pmod{3}$ 인 경우입니다.
- 다음 3×3 패턴을 주기적으로 반복합니다.

C	C	S
C	S	S
C	S	S

- 이 경우도 모든 3×3 부분격자에는 S가 정확히 5개 들어갑니다.

1C. SCSC 마법의 화원

- $N = 3q + 1$ 일 때 이 패턴의 S 개수는

$$5q^2 + q$$

입니다.

- 현재 가능한 범위에서는 $q \leq 4$ 입니다.
- 이때

$$5q^2 + q < \frac{(3q + 1)^2}{2}$$

이므로 전체에서는 C가 더 많습니다.

- 따라서 최종 판정은 다음과 같습니다.
 - $N \geq 16$ 이거나 $3 \mid N$ 이면 NO
 - 그 외에는 위의 주기 3 패턴을 출력

2E. 마법의 상자열기

probability, linearity_of_expectation

출제진 의도 – Medium

- 처음 푼 참가자: **spoiler shuffled by Loreips**, 48분
- 출제자: terrasphere

- 어떤 시점에 다음 마법이 무엇인지는, 아직 남은 마법들 중 균등하게 정해져 있습니다.
- 따라서 남은 마법들을 시도할 때는 소모 마력이 작은 것부터 시도하는 것이 최적입니다.
- 앞으로 마법들을 소모 마력의 오름차순으로 정렬해

$$C_1 \leq C_2 \leq \dots \leq C_N$$

이라고 하겠습니다.

- 이제 각 마법이 몇 번 시도되는지의 기댓값을 구합니다.
- 기댓값의 선형성에 의해, 전체 기댓값은 각 마법의 기여를 따로 더해도 됩니다.
- 마법 i 의 소모 마력이 C_i 일 때, 이 마법이 시도되는 경우는 세 종류로 나눌 수 있습니다.

2E. 마법의 상자 열기



- 첫째, 자기 차례가 왔을 때 성공하기 위해 정확히 한 번 시도됩니다.

$$C_i$$

- 둘째, 더 비싼 마법 j 보다 먼저 시도했다가 실패할 수 있습니다.
- $j > i$ 에 대해, 실제 순서에서 j 가 i 보다 앞에 있으면 C_i 를 한 번 잘못 시도합니다.
- 이 확률은 $\frac{1}{2}$ 이므로, 이 경우의 기여는

$$C_i \cdot \frac{N-i}{2}$$

입니다.

2E. 마법의 상자 열기



- 셋째, 이미 해제한 마법을 다시 시도해야 하는 경우가 있습니다.
- 서로 다른 세 마법 x, a, b 를 생각합니다.
- a 가 b 보다 싸고, 실제 순서가

$$x \rightarrow b \rightarrow a$$

라면, b 를 찾아야 할 때 a 를 먼저 시도했다가 실패합니다.

- 이때 처음부터 다시 가야 하므로, 이미 해제된 x 도 한 번 다시 시도됩니다.

- 고정된 x, a, b 가 $x \rightarrow b \rightarrow a$ 순서로 등장할 확률은 $1/6$ 입니다.
- 마법 i 를 x 로 고정하면, 나머지 $N-1$ 개 중 a, b 를 고르는 방법은 $\binom{N-1}{2}$ 가지입니다.
- 따라서 마법 i 가 다시 시도되는 횟수의 기댓값은

$$\frac{1}{6} \binom{N-1}{2} = \frac{(N-1)(N-2)}{12}$$

입니다.

- 따라서 C_i 의 총 기여는

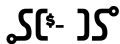
$$C_i \left(1 + \frac{N-i}{2} + \frac{(N-1)(N-2)}{12} \right)$$

입니다.

- 모든 i 에 대해 더하면 답을 얻습니다.

$$\sum_{i=1}^N C_i \left(1 + \frac{N-i}{2} + \frac{(N-1)(N-2)}{12} \right)$$

2E. 마법의 상자 열기



- 정렬한 뒤 위 식을 그대로 계산하면 됩니다.
- 전체 시간 복잡도는 $\mathcal{O}(N \lg N)$ 입니다.

21. 합과 차

dp, combinatorics, case_work

출제진 의도 - **Hard**

- 처음 푼 참가자: **XX**, X분
- 출제자: terrasphere

21. 합과 차

- 가능한 두 연산 모두 제곱합을 정확히 2배로 만듭니다.

$$(A+B)^2 + (A-B)^2 = 2(A^2 + B^2)$$

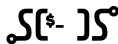
$$(B-A)^2 + (A+B)^2 = 2(A^2 + B^2)$$

- 따라서 길이가 k 인 변환 후에는

$$C^2 + D^2 = 2^k(A^2 + B^2)$$

이어야 합니다.

21. 합과 차



- $(A, B) \neq (0, 0)$ 라면 가능한 문자열의 길이는 많아야 하나입니다.
- 즉,

$$\frac{C^2 + D^2}{A^2 + B^2}$$

가 2의 거듭제곱이어야 하고, 그 지수가 문자열의 길이입니다.

- 이 조건을 만족하지 않으면 답은 바로 -1입니다.

21. 합과 차

- 이제 길이 k 가 정해졌다고 합시다.
- 두 연산을 여러 번 합성해도, 결과는 항상 스케일을 제외하면 네 방향 중 하나입니다.
- 예를 들어 길이가 짝수라면

$$(A, B), \quad (-B, A), \quad (-A, -B), \quad (B, -A)$$

중 하나에 $2^{k/2}$ 가 곱해진 형태입니다.

- 길이가 홀수라면 마지막에 한 번 더 연산이 적용된 형태가 됩니다.
- 따라서 역시 가능한 형태는 네 가지뿐입니다.

$$(A + B, A - B), \quad (B - A, A + B), \\ (-A - B, -A + B), \quad (A - B, -A - B)$$

- 여기에 $2^{(k-1)/2}$ 가 곱해진 형태입니다.

21. 합과 차

- $dp[i][s]$ 를 길이 i 의 문자열 중 결과가 상태 s 가 되는 개수라고 합시다.
- 상태 s 는 방금 본 네 방향 중 하나입니다.
- 처음에는 길이 0의 빈 문자열만 존재하므로

$$dp[0][0] = 1$$

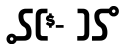
입니다.

- 문자를 하나 붙일 때마다 상태가 바뀌므로, 네 상태 사이의 전이를 미리 계산할 수 있습니다.

- 이 DP를 이용하면 함수 $\text{cnt}(r, X, Y, C, D)$ 를 만들 수 있습니다.
- 이는 현재 값이 (X, Y) 일 때, 길이 r 의 문자열로 (C, D) 에 도달하는 문자열 개수입니다.
- 가능한 형태가 네 가지뿐이므로, 네 형태 중 어느 것과 일치하는지만 확인하면 됩니다.
- 일치하는 상태의 $dp[r][s]$ 가 곧 경우의 수입니다.

- 이제 사전순으로 P 번째 문자열을 복원합니다.
- 현재 남은 길이를 r 이라고 합시다.
- 먼저 다음 문자를 1로 정했을 때 가능한 완성 개수를 셉니다.
- 그 개수가 P 이상이면 1을 선택하고, 아니면 그 개수를 빼고 2를 선택합니다.

21. 합과 차



- 이를 길이가 끝날 때까지 반복하면 원하는 문자열을 얻습니다.
- 만약 전체 가능한 문자열 개수가 P 보다 작으면 -1입니다.
- 길이 0의 문자열이 답이면 EMPTY를 출력합니다.

- $(A, B) = (0, 0)$ 인 경우는 따로 처리해야 합니다.
- 어떤 연산을 해도 계속 $(0, 0)$ 입니다.
- 따라서 $(C, D) \neq (0, 0)$ 이면 불가능합니다.
- 반대로 $(C, D) = (0, 0)$ 이면 모든 문자열이 가능합니다.

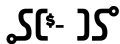
- 원점에서 원점으로 가는 경우에는 길이 순, 사전순으로 모든 이진 문자열을 나열하는 문제가 됩니다.
- 길이 L 이전까지의 문자열 수는

$$1 + 2 + \dots + 2^{L-1} = 2^L - 1$$

입니다.

- 따라서 P 가 속한 길이 L 을 찾고, 그 안에서의 순서를 이진수처럼 해석하면 됩니다.

21. 합과 차



- 한 질의당 시간 복잡도는 $\mathcal{O}(\log P + \log |C^2 + D^2|)$ 입니다.
- 최종 시간 복잡도는 $\mathcal{O}(Q(\log P + \log |C^2 + D^2|))$ 입니다.

1G. 오피

graphs, combinatorics, parity

출제진 의도 – **Hard**

- 처음 푼 참가자: **XX**, X분
- 출제자: sorohue

- 타일의 경계에 길이 있는지 없는지에 집중합니다. 그리고 사용할 수 있는 타일의 형태를 보면, 가로와 세로 중 **한 방향으로만 길의 여부를 바꾸지 않고 다른 방향으로만 길의 여부를 바꾼다**는 사실을 알 수 있습니다.
- 따라서 문제를 각 빈칸에 가로 또는 세로의 방향을 배정해, 빈칸들이 한줄로 붙어있는 모든 블록이 그 양끝에 맞닿은 타일과 길이 잘 이어지도록 만드는 경우의 수를 묻는 문제로 환원할 수 있습니다.

- 타일을 놓을 수 있는 각 **빈칸을 간선으로, 블록을 정점**으로 하는 그래프를 생각합니다.
- 블록의 양끝에 맞닿은 타일로부터, 각 블록에 놓여야 하는 가로 방향 타일의 홀짝성을 알 수 있습니다.
- 각 정점에 0 또는 1이 배정되어 있을 때, 간선을 적절히 선택하고 선택한 간선마다 양끝 정점에 1을 XOR해 모든 정점을 0으로 만드는 방법의 수를 구하는 문제로 다시 환원할 수 있습니다.

- 각 컴포넌트는 독립적입니다. V 개의 정점과 E 개의 간선으로 이루어진 단일 컴포넌트의 경우를 고려합시다.
- 한 간선을 선택하면 두 정점에 1을 더하거나 뺍니다. 따라서 정점에 배정된 값의 합이 홀수인 경우 모두 0으로 만드는 것은 불가능합니다.
- **Claim:** 정점에 배정된 값의 합이 짝수인 경우, 항상 모두 0으로 만들 수 있고, 그 경우의 수는 정점에 배정된 값에 관계없이 모두 같습니다.

- 그래프의 아무 스패닝 트리를 고려합니다.
- 정점에 배정된 값의 합이 짝수인 경우, 스패닝 트리 상의 리프부터 역순으로 각 간선의 선택 여부를 결정하는 식으로 해당 스패닝 트리 상의 간선만으로 모든 정점을 0으로 만드는 **유일한** 방법을 얻어낼 수 있습니다.
- 스패닝 트리에 포함되지 않은 나머지 간선은 선택 여부를 어떻게 정하더라도, 스패닝 트리의 간선들로 이를 수습하는 방법이 유일하게 존재함을 알 수 있습니다.
- 따라서 한 컴포넌트에 타일을 채우는 방법의 수는 $2^{E-(V-1)}$ 가지입니다.

- 격자판의 상황으로 돌아옵니다.
- 블록의 수를 V , 빈칸의 수를 E , 컴포넌트의 수를 F 로 두면 답은 2^{E-V+F} 입니다.
- 계산에 필요한 모든 값을 $\mathcal{O}(NM)$ 에 쉽게 구할 수 있습니다.
- 전체 문제 역시 $\mathcal{O}(NM)$ 에 해결할 수 있습니다.

1F. 칩 연결하기

dp, combinatorics

출제진 의도 – **Hard**

- 처음 푼 참가자: **queued_q**, 123분
- 출제자: sorohue

- 빈 슬롯을 기준으로 0번부터 N 번까지의 블록을 고려합니다. i 번 블록에는 S_i 개의 S-칩과 C_i 개의 C-칩이 들어 있고, 이 칩들은 $i-1$ 번 i 번, $i+1$ 번 블록의 칩들과 연결될 수 있습니다.
- $d[i][s]$ 를 [i 번 이하의 블록에 들어있는 칩들을 이용해 i 번 미만의 블록에 들어있는 모든 칩을 연결하고 i 번 블록에 s 개의 S-칩을 남기는 경우의 수]로 정의합니다. 남은 S-칩의 개수를 알면 남은 C-칩의 개수 역시 간단히 알 수 있습니다.
- $d[i-1][s']$ 에서 $d[i][s]$ 로 전이하는 것은 $i-1$ 번 블록에 남아있는 칩들을 i 번 블록의 칩과 모두 연결하고 남은 S-칩의 수가 s 개가 될 때까지 i 번 블록의 칩들을 서로 연결해주는 방법의 수를 구하는 것으로 할 수 있습니다. 팩토리얼 및 그 역원 등을 전처리하면 이 값은 $\mathcal{O}(1)$ 에 계산할 수 있습니다.

- $d[i][s]$ 에서 s 의 값으로 유효한 것은 i 번 블록에 포함된 $S_i + 1$ 이하입니다. 따라서 전체 상태의 수는 $\mathcal{O}(N + M)$ 개입니다. 각 상태는 다음 블록의 상태들로만 전이할 수 있으므로 한 상태에서의 모든 전이를 계산하는 데는 $\mathcal{O}(M)$ 이 걸립니다.
- 총 시간복잡도 $\mathcal{O}(M(N + M))$ 에 문제를 해결할 수 있습니다.
- ACL의 convolution을 이용해 $i - 1$ 번 블록의 모든 상태에서 i 번 블록의 모든 상태로의 전파를 $\mathcal{O}((S_{i-1} + S_i) \lg(N + M))$ 에 처리할 수도 있습니다. 따라서 문제를 $\mathcal{O}((N + M) \lg(N + M))$ 에도 해결할 수 있으나, 이를 강제하지는 않았습니다.

1J/2L/3J. 채널톡 워크플로우

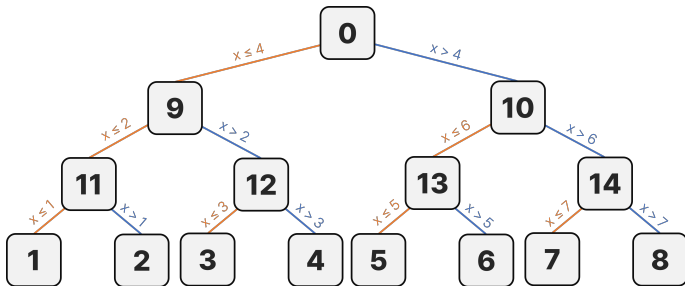
constructive

출제진 의도 - **Hard**

- **Div. 1** — 처음 푼 참가자: **ibm2004**, 45분
- **Div. 2** — 처음 푼 참가자: **#include <furries>**, 230분
- **Div. 3** — 처음 푼 참가자: **XX**, X분
- 출제자: sorohue

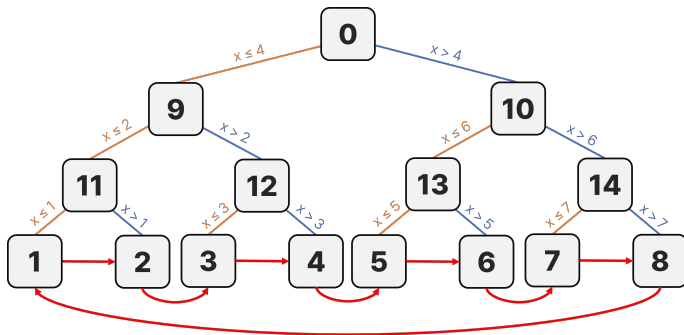
- 각 모듈이 많아야 하나의 질문에 답변할 수 있습니다. 동시에 모든 질문에 답변하려면 일단 모든 질문을 서로 다른 모듈에 집어넣을 시간이 필요합니다.
- 이진 트리 같은 구조를 이용해 질문 집합을 계속 절반씩 나누는 게 가장 빠릅니다. 이를 위해 적어도 $\lceil \lg N \rceil + 1$ 초가 필요합니다. $K \leq \lceil \lg N \rceil$ 인 경우 가능한 해가 없습니다.
- **Claim:** 이외의 경우 입력 조건인 $K \leq N$ 상에서 항상 해 구성이 가능합니다.

- Claim의 증명은 실제로 해를 구성하는 식으로 할 수 있습니다.
- 우선 이진 트리를 만들어서 $\lceil \lg N \rceil + 1$ 초에 모든 질문이 리프 모듈에 도달하도록 만듭시다.

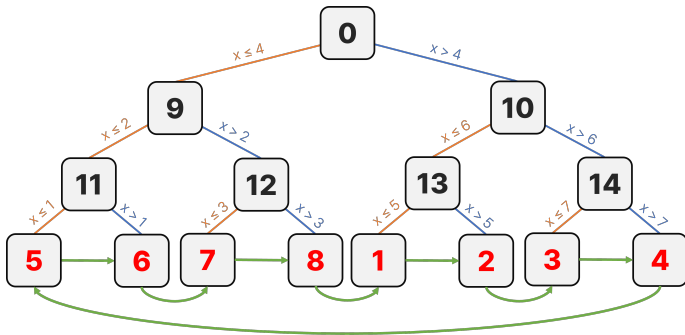


$N = 8; K = 4$ 일 때의 예시

- $K > \lceil \lg N \rceil + 1$ 인 경우 각 질문이 올바른 리프 모듈에 도달하기까지의 시간을 지연시켜야 합니다.
- 각 리프 모듈의 G, L 값을 모두 옆 리프 모듈 번호로 설정해 사이클을 만들어 줍니다.



- 이제 t 초를 지연해야 하는 상황을 생각합시다. 그러면 단순히 리프 모듈의 순서를 사이클 방향으로 t 칸만큼 돌려주는 것으로 모든 질문에 t 초의 지연을 만들어낼 수 있음을 알 수 있습니다.



$N = 8; K = 8$ 일 때의 예시

- 이 방법으로 적어도 최대 $N - 1$ 초의 시간을 벌 수 있습니다. 입력 조건상 필요한 최대 지연은 $N - 1 - \lceil \lg N \rceil$ 초이므로 항상 필요한 만큼의 시간을 벌 수 있습니다.
- $N = 1000$ 일 때 포화 이진 트리의 형태로 해를 구성하려면 최대 2047개의 모듈이 필요하고 이는 모듈 개수 제한인 $M \leq 2048$ 에 들어갑니다. 따라서 항상 딱 맞는 높이의 포화 이진 트리 형태로 해를 구성하는 식으로 구현을 단순화할 수 있습니다.

2K. 찬란한 연방의 놀이터에 대해

rerooting

출제진 의도 - Hard

- 처음 푼 참가자: **XX**, X분
- 출제자: sorohue

- 편의상 포인트를 정점으로, 통로를 간선으로 서술합니다.
- 정점을 미끄럼틀로 연결된 사이클로 분할합니다.
- 사이클에 포함된 모든 정점은 임의의 정점 s 로부터의 최대 거리가 서로 같습니다.
- 사이클 별로 해당 사이클에 포함된 정점 중 어떤 정점과 가장 멀리 있는 정점 사이의 거리를 빠르게 관리할 수 있으면 됩니다.

- 사이클에 포함된 정점들을 모두 포함하는 가장 작은 서브트리를 고려합니다. 이러한 서브트리는 유일하게 존재합니다. 이때 서브트리의 모든 리프 노드는 사이클에 포함되는 정점임을 알 수 있습니다.
- 트리의 성질을 고려하면, 전체 트리 안의 임의의 정점 u 에 대해 u 와 가장 멀리 떨어진 서브트리 안의 정점은 그 서브트리의 지름의 양끝 정점 v_1, v_2 중 하나입니다. 지름의 끝점은 리프 노드이므로 v_1 과 v_2 는 모두 사이클에 속하는 점입니다.
- 그러면 v_1 과 v_2 까지의 거리 중 더 긴 걸 고르면 되겠네요. 각 정점마다 이 계산을 하면 최악의 경우 $\mathcal{O}(N^2)$ 에 문제를 해결할 수 있습니다.
- ? 그냥 모든 정점 쌍 사이의 거리를 재도 $\mathcal{O}(N^2)$ 에 풀리는데요??

- v_1 과 v_2 중 더 먼 것까지 가는 경로를 생각합니다. 그러면 그 경로가 반드시 지나는 지점이 있는데, 바로 서브트리의 지름의 **중심**입니다.
 - 경로가 지름과 처음으로 만나는 정점 z 를 고려합니다. 경로가 지름의 중심을 지나지 않는 경우의 경로를 $u - z - v_1$ 으로 표현하면, $z - v_1$ 의 거리보다 $z - v_2 = (v_1 - v_2) \setminus (z - v_1)$ 의 거리가 더 길겠죠.
- 따라서 사이클에서 가장 먼 정점까지의 거리를 서브트리의 지름의 중심까지의 거리와 서브트리의 반지름의 합으로 분해할 수 있습니다.
- 서브트리의 반지름의 길이는 불변하니 각 서브트리의 중심까지의 거리 합만 잘 관리해주면 됩니다.

- 서브트리의 중심은 정점이 될 수도 있고 간선의 한가운데가 될 수도 있습니다. 이를 처리하기 위해 모든 간선 가운데에 정점을 하나씩 추가로 배치합니다. 이제 모든 필요한 서브트리의 중심이 정점으로 표현됩니다.
- 이제 우리는 사이클에 의해 정해진 몇 개의 정점들과 임의의 정점 u 사이의 거리의 합을 모든 $u = 1, 2, \dots, N$ 에 대해 구해 문제를 해결할 수 있습니다.
- 이 작업은 리루팅으로 쉽게 할 수 있습니다.
- 총 시간 복잡도는 $\mathcal{O}(N \lg N)$ 입니다.

1B. 스시|스시 유적

string, Z, segtree

출제진 의도 - **Hard**

- 처음 푼 참가자: **xx**, X분
- 출제자: terrasphere

- T 에서 어떤 연속한 부분을 지운다고 합시다.
- 그러면 남는 문자열은 T 의 prefix와 T 의 suffix를 이어 붙인 꼴입니다.
- 즉, 어떤 S 의 부분문자열이 유효하려면, 앞부분은 T 의 prefix와 같고 뒷부분은 T 의 suffix와 같아야 합니다.
- 이하에서는 인덱스를 0부터 시작한다고 생각하겠습니다.

- S 의 구간 $[s, e]$ 가 삭제 후의 문자열로 쓰인다고 합시다.
- 이 구간을 어떤 위치 c 에서 나눕니다.

$$S[s..c] + S[c+1..e]$$

- 왼쪽은 T 의 prefix, 오른쪽은 T 의 suffix와 매칭됩니다.

- F_s 를 T 와 $S[s..]$ 의 LCP 길이라고 합시다.
- B_e 를 T 와 $S[..e]$ 의 공통 suffix 길이라고 합시다.
- F_s 는 $T\#S$ 의 Z-array로 구할 수 있습니다.
- B_e 는 문자열을 뒤집은 뒤 같은 방식으로 구할 수 있습니다.

1B. 스시스시 유적

- 고정된 구간 $[s, e]$ 에 대해 가능한 분할 위치 c 를 봅시다.
- 왼쪽이 prefix와 맞으려면 $c \leq s + F_s - 1$ 이어야 합니다.
- 오른쪽이 suffix와 맞으려면 $c \geq e - B_e$ 이어야 합니다.
- 따라서 가능한 c 는 다음 두 구간의 교집합입니다.

$$[s - 1, s + F_s - 1] \cap [e - B_e, e]$$

- 삭제하는 길이는 1 이상이어야 하므로, 남은 문자열의 길이는 $M - 1$ 이하여야 합니다.
- 즉, S 의 구간 $[s, e]$ 에 대해

$$1 \leq e - s + 1 \leq M - 1$$

이어야 합니다.

- 따라서 시작점 s 를 고정하면 가능한 끝점은

$$s \leq e \leq s + M - 2$$

입니다.

- 끝점 e 하나는 분할 위치 c 에 대해 구간

$$[e - B_e, e]$$

를 기여합니다.

- 시작점 s 에서는 분할 위치 c 가

$$[s - 1, s + F_s - 1]$$

안에 있어야 합니다.

- 따라서 활성화된 끝점들의 구간을 모아 두고, 시작점마다 이 구간 합을 질의하면 됩니다.

- 시작점 s 를 왼쪽에서 오른쪽으로 움직입니다.
- 자료구조에는 현재 가능한 끝점 $s \leq e \leq s + M - 2$ 의 구간 $[e - B_e, e]$ 를 넣어 둡니다.
- 답에는 구간 $[s - 1, s + F_s - 1]$ 의 합을 더합니다.
- 이후 끝점 $e = s$ 를 제거하고, $e = s + M - 1$ 을 새로 추가합니다.

- 필요한 연산은 구간에 $+1$ 또는 -1 을 더하는 것과, 구간 합을 구하는 것입니다.
- 이는 Fenwick tree 두 개 또는 lazy segment tree로 처리할 수 있습니다.
- 분할 위치 $c = s - 1$ 이 나올 수 있으므로, 구현에서는 인덱스에 적당한 shift를 더해 관리합니다.
- $M = 1$ 이면 지울 수 있는 길이 K 가 없으므로 답은 0입니다.

- F_s 와 B_e 는 Z-algorithm으로 $\mathcal{O}(N + M)$ 에 계산합니다.
- 이후 sweep 과정에서 각 끝점 구간은 상수 번 추가되고 제거됩니다.
- 전체 시간 복잡도는 $\mathcal{O}((N + M) \lg N)$ 입니다.

11. 트리와 게임과 퀴리

tree, game_theory

출제진 의도 - **Hard**

- 처음 푼 참가자: **jinhan814**, 126분
- 출제자: terrasphere

11. 트리와 게임과 퀴리

- 현재 루트를 R 이라고 합시다.
- 각 정점 i 에 놓인 돌의 개수를 $C_i = V_i W_i$ 라고 둡니다.
- 다음 값을 생각합시다.

$$X_R = \bigoplus_{d(R,i) \text{ odd}} C_i$$

- **Claim:** 현재 플레이어가 이길 조건은 $X_R \neq 0$ 입니다.

- 먼저 $X_R = 0$ 인 상태를 봅시다.
- 돌 k 개를 어떤 정점에서 부모로 옮기면, 홀수 거리 정점의 돌 개수 중 정확히 하나가 바뀝니다.
- 출발 정점이 홀수 거리라면 그 정점의 돌 개수가 $C \rightarrow C - k$ 로 바뀝니다.
- 출발 정점이 짝수 거리라면 그 부모가 홀수 거리이고, 부모의 돌 개수가 $C \rightarrow C + k$ 로 바뀝니다.
- 따라서 이동 후에는 반드시 $X_R \neq 0$ 이 됩니다.

11. 트리와 게임과 쿼리

- 이제 $X_R \neq 0$ 인 상태를 봅시다.
- 일반적인 Nim처럼, X_R 의 가장 높은 비트를 가진 홀수 거리 정점 v 를 고릅니다.
- $C'_v = C_v \oplus X_R$ 라고 두면 $C'_v < C_v$ 입니다.
- 정점 v 에서 부모로 $C_v - C'_v$ 개의 돌을 옮기면, 홀수 거리 정점들의 xor가 0이 됩니다.
- 홀수 거리 정점은 루트가 아니므로 이 이동은 항상 가능합니다.

11. 트리와 게임과 퀴리

- 따라서 현재 게임의 승패는 오직

$$X_R = \bigoplus_{d(R,i) \text{ odd}} V_i W_i$$

만으로 결정됩니다.

- $X_R = 0$ 이면 Lulu가 이기고, $X_R \neq 0$ 이면 Terra가 이깁니다.
- 이제 남은 문제는 퀴리마다 이 xor 값을 빠르게 관리하는 것입니다.

11. 트리와 게임과 쿼리

- 트리를 처음 루트 1로 고정하고 깊이를 계산합니다.
- 현재 루트가 R 일 때,

$$d(R, i) \equiv \text{depth}(R) + \text{depth}(i) \pmod{2}$$

입니다.

- 따라서 $\text{depth}(R)$ 이 짝수이면 기존 깊이가 홀수인 정점들을 보고, 홀수이면 기존 깊이가 짝수인 정점들을 보면 됩니다.

- 다음 두 값을 관리합니다.

$$X_0 = \bigoplus_{\substack{\text{depth}(i) \text{ even} \\ W_i=1}} V_i, \quad X_1 = \bigoplus_{\substack{\text{depth}(i) \text{ odd} \\ W_i=1}} V_i$$

- 현재 루트 R 의 깊이가 짝수이면 필요한 값은 X_1 입니다.
- 현재 루트 R 의 깊이가 홀수이면 필요한 값은 X_0 입니다.

11. 트리와 게임과 쿼리

- 쿼리 2 z 는 루트만 z 로 바꾸는 연산입니다.
- X_0, X_1 자체는 처음 루트 1 기준의 깊이 홀짝성으로 관리됩니다.
- 따라서 루트 변경 쿼리에서는 현재 루트 번호만 바꾸면 됩니다.
- 이후 $depth(R)$ 의 홀짝성에 따라 X_0 또는 X_1 을 확인합니다.

- 쿼리 1 $x\ y$ 는 경로 $P(x, y)$ 위의 모든 W_i 를 뒤집습니다.
- 어떤 정점의 W_i 가 $0 \leftrightarrow 1$ 로 바뀌면, 관리 중인 xor 값에는 V_i 가 한 번 xor됩니다.
- 따라서 경로 위 정점들의 V_i xor를 깊이 홀짝성별로 구해 X_0, X_1 에 반영하면 됩니다.

11. 트리와 게임과 쿼리

- 루트 1에서 각 정점까지의 prefix xor를 저장합니다.
- $P_0(v)$ 는 1에서 v 까지의 짝수 깊이 정점들의 V_i xor입니다.
- $P_1(v)$ 는 1에서 v 까지의 홀수 깊이 정점들의 V_i xor입니다.
- 이 두 배열과 LCA를 이용하면 경로의 홀짝별 xor를 구할 수 있습니다.

11. 트리와 게임과 쿼리

- $l = \text{lca}(x, y)$ 라고 합시다.
- 짝수 깊이 정점들의 경로 xor는

$$P_0(x) \oplus P_0(y)$$

에서 시작합니다.

- 이 값에는 l 이 두 번 빠져 있으므로, $\text{depth}(l)$ 이 짝수라면 V_l 을 한 번 더 xor합니다.
- 홀수 깊이도 같은 방식으로 계산합니다.

11. 트리와 게임과 쿼리

- 경로 $P(x, y)$ 의 짝수 깊이 xor를 Y_0 , 홀수 깊이 xor를 Y_1 이라고 합시다.
- 쿼리 $1 \times y$ 는

$$X_0 \leftarrow X_0 \oplus Y_0, \quad X_1 \leftarrow X_1 \oplus Y_1$$

로 처리됩니다.

- 이후 현재 루트에 맞는 xor 값이 0인지 확인하면 됩니다.

11. 트리와 게임과 쿼리

- 전처리로 깊이, LCA, 홀짝별 prefix xor를 구합니다.
- 쿼리 1 x, y 는 경로의 홀짝별 xor를 구해 X_0, X_1 에 반영합니다.
- 쿼리 2 z 는 현재 루트만 z 로 바꿉니다.
- 매 쿼리 후, 현재 루트 기준의 홀수 거리 xor가 0이면 Lulu, 아니면 Terra가 이깁니다.
- 전체 시간 복잡도는 $\mathcal{O}((N + Q) \lg N)$ 입니다.

1K. 롤케이크 보관

dp, CHT

출제진 의도 - **Hard**

- 처음 푼 참가자: **jhwest2**, 105분
- 출제자: terrasphere

1K. 롤케이크 보관

- 최종적으로 롤케이크가 여러 조각으로 나뉘었다고 합시다.
- 각 조각에서는 오른쪽 끝 조각만 보이므로, 아름다움은 각 조각의 오른쪽 끝 위치 i 에 대한 V_i 의 합입니다.
- 중요한 것은 자르는 데 드는 힘입니다.
- 최종 조각들의 길이가 $a_1, a_2, \dots, a_m$ 이라면, 전체 힘은

$$\sum_{1 \leq p < q \leq m} a_p a_q$$

입니다.

- 즉, 자르는 순서는 중요하지 않습니다.

1K. 롤케이크 보관

- $dp[i]$ 를 1번부터 i 번 조각까지 보관했을 때 얻을 수 있는 최대값으로 둡니다.
- 마지막 조각이 $j + 1$ 번부터 i 번까지라면, 마지막 조각의 길이는 $i - j$ 입니다.
- 이때 마지막 조각의 오른쪽 끝은 i 번이므로 아름다움에 v_i 가 더해집니다.
- 또한 앞쪽 길이 j 와 마지막 조각 길이 $i - j$ 사이에서 새로 생기는 힘은

$$j(i - j)$$

입니다.

- 1K. 롤케이크 보관
- 따라서 다음 점화식을 얻습니다.

$$dp[i] = V_i + \max_{\max(0, i-K) \leq j < i} (dp[j] - j(i-j))$$

- 식을 정리하면

$$dp[i] = V_i + \max_{\max(0, i-K) \leq j < i} (dp[j] + j^2 - ij)$$

입니다.

- 여기서 i 를 변수로 보면, 각 j 는 하나의 직선

$$y = -jx + (dp[j] + j^2)$$

을 나타냅니다.

- 즉, 문제는 다음 형태가 됩니다.
 - j 에 해당하는 직선을 추가한다.
 - 현재 i 에서 사용할 수 있는 직선들 중 최댓값을 구한다.
- 직선의 기울기 $-j$ 는 단조롭게 변합니다.
- 쿼리하는 $x = i$ 역시 단조롭게 증가합니다.
- 따라서 덱을 이용한 Convex Hull Trick을 적용할 수 있습니다.

1K. 롤케이크 보관

- 다만 사용할 수 있는 j 는 항상

$$i - K \leq j < i$$

를 만족해야 합니다.

- 이를 길이 K 단위의 블록으로 나누어 이 문제를 해결할 수 있습니다.
- 현재 블록을 $[S, E]$ 라고 하면, $i \in [S, E]$ 에서 가능한 j 는 두 종류입니다.

$$i - K \leq j < S$$

인 이전 블록의 후보와,

$$S \leq j < i$$

인 현재 블록 안의 후보입니다.

1K. 롤케이크 보관

- 이전 블록의 후보는 i 를 오른쪽에서 왼쪽으로 보며 처리합니다.
- 이때 가능한 후보 집합

$$i - K \leq j < S$$

은 점점 커지기만 합니다.

- 한편 각 j 는 직선

$$y = -jx + (dp[j] + j^2)$$

으로 볼 수 있습니다.

- i 가 감소할수록 새로 추가되는 j 도 감소하므로, 직선의 기울기 $-j$ 는 증가합니다.
- 즉, 쿼리 위치와 직선의 기울기가 모두 단조적입니다.

- 현재 블록의 후보는 i 를 왼쪽에서 오른쪽으로 보며 처리합니다.
- 이때 가능한 후보 집합

$$S \leq j < i$$

도 점점 커지기만 합니다.

- $dp[i]$ 를 계산한 뒤, $j = i$ 에 해당하는 직선을 추가하면 됩니다.
- 이번에는 i 가 증가할수록 새로 추가되는 j 도 증가하므로, 직선의 기울기 $-j$ 는 감소합니다.
- 역시 쿼리 위치와 직선의 기울기가 모두 단조적이므로 CHT를 사용할 수 있습니다.

- 최종 DP값은 이전 블록 후보의 최댓값과, 현재 블록 후보의 최댓값중 큰 값을 사용하여 업데이트합니다.
- 이를 통해 삭제 연산이 없는 단조 CHT만을 사용해 문제를 해결할 수 있습니다.
- 전체 시간 복잡도는 $\mathcal{O}(L)$ 입니다.

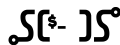
1E. “저는 삼각기둥이 되고 싶어요”

geometry, half_plane_intersection

출제진 의도 – **Challenging**

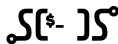
- 처음 푼 참가자: **XX**, X분
- 출제자: sungjae0506

1E. "저는 삼각기둥이 되고 싶어요"



- 밑면 삼각형의 넓이를 A , 둘레를 P , 삼각기둥의 높이를 h 라 합시다.
- 부피는 $V = Ah$, 겉넓이는 $S = 2A + Ph$ 이므로, $\frac{V}{S} = \frac{Ah}{2A + Ph}$ 입니다.
- 고정된 밑면에 대해 이 값은 h 가 커질수록 증가합니다.
- 모든 점을 포함하려면 $h \geq z_{\max} - z_{\min}$ 이어야 하므로, 높이는 $H = z_{\max} - z_{\min}$ 입니다.

1E. "저는 삼각기둥이 되고 싶어요"



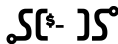
- 높이는 H 로 고정되었으므로, xy 평면에서 밑면 삼각형만 찾으면 됩니다.
- 고정된 H 에 대해 $\frac{V}{S} = \frac{AH}{2A+PH} = \frac{H(A/P)}{2(A/P)+H}$ 입니다.
- 따라서 $\frac{V}{S}$ 를 최소화하는 것은 $\frac{A}{P}$ 를 최소화하는 것과 같습니다.
- Convex hull 내부 점은 의미가 없으므로, xy 좌표의 convex hull을 포함하는 삼각형만 고려합니다.

1E. "저는 삼각기둥이 되고 싶어요"



- 삼각형의 내접원 반지름을 r 이라 하면 $A = \frac{Pr}{2}$, 즉 $A/P = r/2$ 입니다.
- 따라서 A/P 최소화는 내접원 반지름 r 최소화와 하는 것과 같습니다.
- 이제 convex hull을 포함하는 삼각형 중 내접원 반지름이 최소인 것을 찾는 문제로 바뀝니다.

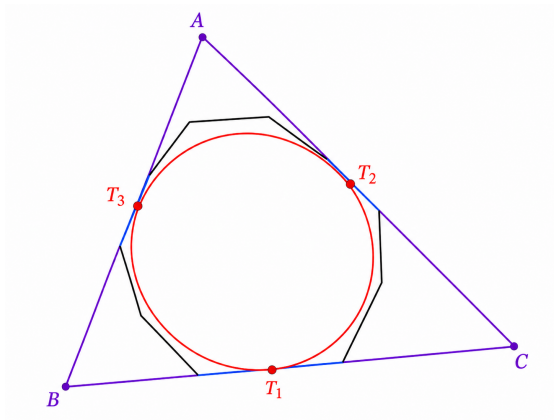
1E. “저는 삼각기둥이 되고 싶어요”



- Convex hull을 포함하는 삼각형의 내접원 반지름은 convex hull의 최대 내접원 반지름보다 작을 수 없습니다.
- 또한 최대 내접원에 접하는 세 변을 연장한 직선은 convex hull을 포함하는 삼각형을 만듭니다.
- 따라서 최대 내접원에 접하는 세 변을 연장해서 만든 삼각형은 내접원 반지름이 최소인 삼각형입니다.
- 최대 내접원의 중심으로부터 가장 가까운 세 변이 최대 내접원에 접하는 변들입니다.

1E. "저는 삼각기둥이 되고 싶어요"

SCSC



1E. "저는 삼각기둥이 되고 싶어요"



- Convex hull 계산은 $O(N \log N)$ 입니다.
- Convex hull 크기를 M 이고 좌표의 범위가 K 이라 하면, HPI와 이분 탐색으로 $O(M \log M \log K)$ 에 최대 내접원의 중심을 구할 수 있습니다.
- 또는 3D Randomized Incremental LP 로 평균 $O(M)$ 에 최대 내접원의 중심을 구할 수도 있습니다.

1L. 비전 마법사 루루

`sqrt_decomposition, dp_tree, lca`

출제진 의도 – **Challenging**

- 처음 푼 참가자: **XX**, X분
- 출제자: terrasphere

- 쿼리에서 구해야 하는 값은

$$\sum_{x \in P(u, v)} \sum_{r=1}^N W_r d(r, x)$$

입니다.

- 각 정점 x 에 대해

$$F(x) = \sum_{r=1}^N W_r d(r, x)$$

를 알고 있다면, 답은 경로 $P(u, v)$ 위의 $F(x)$ 합입니다.

- 먼저 모든 w_r 가 고정되어 있다고 생각합니다.
- 목표는 모든 정점 x 에 대해 $F(x) = \sum_r w_r d(r, x)$ 를 구하는 것입니다.
- 이는 트리에서의 rerooting DP로 계산할 수 있습니다.
- 이후 루트에서 각 정점까지의 F prefix 합을 저장하면, 경로 합도 LCA로 구할 수 있습니다.

- 루트를 1로 잡습니다.
- cnt_v 를 v 의 서브트리에 있는 위력의 합이라고 합시다.
- $down_v$ 를 서브트리 안의 정점들만 보았을 때의 거리 기여,

$$down_v = \sum_{r \in subtree(v)} W_r d(v, r)$$

로 둡니다.

- cnt_v 와 $down_v$ 는 아래에서 위로 한 번에 계산됩니다.

- 이제 $F(1) = down_1$ 입니다.
- 간선 $p - c$ 를 따라 기준 정점을 p 에서 c 로 옮깁니다.
- c 의 서브트리 안 정점들은 거리가 1 줄고, 나머지 정점들은 거리가 1 늘어납니다.
- 전체 위력의 합을 T 라고 하면

$$F(c) = F(p) + T - 2cnt_c$$

입니다.

- 모든 정점의 $F(x)$ 를 구한 뒤, 루트에서 각 정점까지의 prefix 합을 저장합니다.
- $l = \text{lca}(u, v)$ 라고 하면, 경로 $P(u, v)$ 위의 F 합은

$$\text{pref}_u + \text{pref}_v - 2\text{pref}_l + F(l)$$

입니다.

- 따라서 정적 상태에서는 쿼리를 빠르게 처리할 수 있습니다.

- 문제는 중간에 위력 증가 연산이 섞여 있다는 점입니다.
- 매 업데이트마다 모든 $F(x)$ 를 다시 계산할 수는 없습니다.
- 업데이트를 버킷 단위로 모아 두고, 버킷이 시작될 때만 정적 정보를 다시 계산합니다.
- 버킷 안의 업데이트는 아직 $F(x)$ 에 반영하지 않습니다.

- 쿼리 2 $u\ v$ 의 답은 두 부분으로 나눕니다.
- 첫째, 버킷 시작 시점에 rebuilt된 정적 부분입니다.
- 둘째, 현재 버킷 안에서 새로 생긴 업데이트들의 기여입니다.

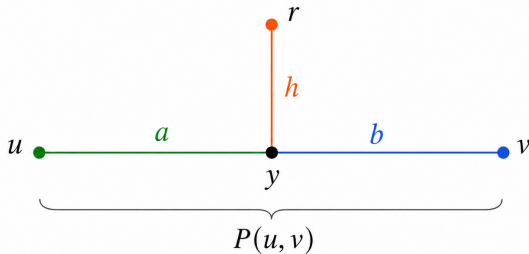
$$\text{answer} = \text{rebuilt path sum} + \text{pending updates}$$

- 버킷 안의 업데이트 하나를 봅시다.
- 정점 r 의 위력이 w 만큼 증가했다면, 쿼리 2 u, v 에 추가되는 값은

$$w \sum_{x \in P(u, v)} d(r, x)$$

입니다.

- 따라서 고정된 r, u, v 에 대해 $\sum_{x \in P(u, v)} d(r, x)$ 를 빠르게 구하면 됩니다.



$$\sum_{x \in P(u, v)} d(r, x) = h(a + b + 1) + \frac{a(a + 1)}{2} + \frac{b(b + 1)}{2}$$

$$a = d(u, y), \quad b = d(v, y), \quad h = d(r, y)$$

- 그림에서 y 는 r 에서 경로 $P(u, v)$ 에 가장 가까운 정점입니다.
- $a = d(u, y)$, $b = d(v, y)$, $h = d(r, y)$ 라고 합시다.
- 모든 $x \in P(u, v)$ 에 대해 $d(r, x) = h + d(y, x)$ 입니다.
- 따라서 경로의 양쪽에서 생기는 거리는 두 개의 삼각수로 계산됩니다.

- 경로 $P(u, v)$ 의 길이는 $a + b$ 이고, 경로 위 정점 수는 $a + b + 1$ 입니다.
- 따라서 h 는 $a + b + 1$ 번 더해집니다.
- 나머지는 y 에서 u 방향으로 $1 + \dots + a$, v 방향으로 $1 + \dots + b$ 입니다.

$$\sum_{x \in P(u, v)} d(r, x) = h(a + b + 1) + \frac{a(a + 1)}{2} + \frac{b(b + 1)}{2}$$

- 이제 a, b, h 를 구하면 됩니다.
- 세 거리

$$A = d(u, v), \quad B = d(r, u), \quad C = d(r, v)$$

를 LCA로 구합니다.

- 그림에서

$$A = a + b, \quad B = h + a, \quad C = h + b$$

입니다.

- 앞의 세 식을 풀면

$$a = \frac{A+B-C}{2}, \quad b = \frac{A+C-B}{2}, \quad h = \frac{B+C-A}{2}$$

입니다.

- 즉, 업데이트 하나의 기여는 LCA 몇 번과 삼각수 계산으로 구할 수 있습니다.
- 이 값에 업데이트된 위력 w 를 곱해 답에 더합니다.

- 전체 풀이를 정리합시다.
- 버킷이 시작될 때, 지금까지 확정된 위력으로 모든 $F(x)$ 를 다시 계산합니다.
- 쿼리 2 $u\ v$ 가 들어오면 먼저 rebuild된 F 의 경로 합을 구합니다.
- 이후 현재 버킷 안의 업데이트들을 하나씩 보며, 방금의 거리합 공식을 더합니다.

- 버킷 안에 업데이트가 최대 B 개 있도록 나눕니다.
- 각 쿼리는 현재 버킷 안의 업데이트를 최대 B 개 확인합니다.
- rebuild는 버킷마다 한 번 수행됩니다.
- 전체 시간 복잡도는 $\mathcal{O}(BQ + NQ/B)$ 입니다.

- 제공된 분할 대신, 업데이트의 영향을 즉시 자료구조에 반영하는 풀이도 가능합니다.
- 핵심은 거리 공식을 다시 쓰는 것입니다.

$$d(r, x) = dep(r) + dep(x) - 2dep(lca(r, x))$$

- 앞의 두 항은 전역 변수로 관리할 수 있습니다.
- 어려운 부분은 $lca(r, x)$ 항의 경로 합을 처리하는 것입니다.

- 현재까지의 업데이트에 대해 $S_0 = \sum_r W_r$, $S_1 = \sum_r W_r dep(r)$ 를 관리합니다.
- 또한 $G(x) = \sum_r W_r dep(lca(r, x))$ 라고 둡니다.
- 거리식 $d(r, x) = dep(r) + dep(x) - 2dep(lca(r, x))$ 를 경로 위에서 모두 더하면,

$$ans = |P|S_1 + \left(\sum_{x \in P} dep(x) \right) S_0 - 2 \sum_{x \in P} G(x)$$

입니다.

- 따라서 남은 문제는 $G(x)$ 의 경로 합을 동적으로 관리하는 것입니다.
- 업데이트 (r, w) 는 모든 정점 x 에 대해

$$G(x) += w \cdot dep(lca(r, x))$$

를 적용하는 것과 같습니다.

- HLD로 루트에서 r 까지의 경로를 heavy chain 단위로 나눕니다.
- 한 heavy chain 안에서는 필요한 값이 깊이에 대한 일차식 또는 상수식으로 나타납니다.

1L. 비전 마법사 루루

- 각 heavy chain 위에 세그먼트 트리를 올립니다.
- 세그먼트 트리는 구간에

$$A \cdot dep(x) + B$$

꼴의 값을 lazy로 더할 수 있게 만듭니다.

- 그러면 업데이트 하나는 $\mathcal{O}(\lg^2 N)$ 에 처리됩니다.
- 쿼리 경로도 HLD로 나눈 뒤, 각 구간의 값을 더하면 됩니다.

- 이 HLD 풀이의 시간 복잡도는 $\mathcal{O}((N+Q)\lg^2 N)$ 입니다.
- 같은 아이디어를 Link-Cut Tree로 구현하면, 경로 갱신과 경로 질의를 amortized $\mathcal{O}(\lg N)$ 에 처리할 수 있습니다.
- 따라서 LCT를 사용하면 전체 시간 복잡도는 $\mathcal{O}((N+Q)\lg N)$ 입니다.

1D. Mobilint 텐서 스케줄링 (ARIES)

tree, greedy

출제진 의도 – Challenging

- 처음 푼 참가자: **ibm2005**, 63분
- 출제자: lycoris1600

1D. Mobilint 텐서 스케줄링 (ARIES)

- 리프가 아닌 각 노드를 하나의 작업으로 봅니다.
- i 번 작업은 모든 자식 텐서가 메모리에 있을 때 수행할 수 있습니다.
- 현재 메모리 사용량이 C 라면, 수행 순간 피크 후보는 $C + w_i$ 입니다.
- $\text{ch}(i)$ 를 i 의 자식 집합이라고 하면, 수행 후 메모리 변화량은 $\Delta_i = w_i - \sum_{j \in \text{ch}(i)} w_j$ 입니다.
- 따라서 가능한 위상 정렬 중 다음 값을 최소화하면 됩니다.

$$\max_i \left(w_i + \sum_{\text{ord}(j) < \text{ord}(i)} \Delta_j \right)$$

1D. Mobilint 텐서 스케줄링 (ARIES)

- 같은 내용을 free memory 관점으로 바꾸겠습니다.
- 피크 한도를 X , 초기 리프 텐서 크기의 합을 S 라고 합시다.
- 그러면 초기 free memory는 $X - S$ 입니다.
- 현재 free memory가 F 일 때, i 번 작업은 $F \geq w_i$ 이면 수행할 수 있습니다.
- 수행 뒤 free memory는

$$F + \sum_{j \in \text{ch}(i)} w_j - w_i$$

가 됩니다.

- 따라서 각 작업은 다음 두 값으로 표현됩니다.

$$(R_i, G_i) = \left(w_i, \sum_{j \in \text{ch}(i)} w_j - w_i \right)$$

- R_i 는 필요한 free memory, G_i 는 작업 후 free memory 변화량입니다.
- 이제 트리 위 선후관계가 있는 작업들에 대해, 초기 free memory를 최소화하는 문제가 되었습니다.

- Faker's algorithm은 트리의 위상 정렬을 최적화하는 알고리즘입니다.
- 두 인접한 블록의 순서를 바꾸는 exchange argument로, 어떤 자식 블록이 부모 블록과 붙어 나와도 되는 상황을 찾습니다.
- 그런 부모-자식 블록을 하나로 contract하는 과정을 반복해 최적 순서를 구합니다.
- 즉, 각 작업이 필요한 free memory와 free memory 변화량을 가질 때, 선후관계를 지키며 필요한 초기 free memory를 최소화하는 Faker's algorithm을 이용해서 해결할 수 있습니다.

- Faker's algorithm을 적용해 필요한 초기 free memory의 최솟값 F 를 구합니다.
- 초기 리프 텐서 크기의 합을 S 라고 하면 답은 $S + F$ 입니다.
- 이 값이 M 을 넘으면 OOM을 출력하고, 그렇지 않으면 그 값을 출력합니다.
- 전체 시간 복잡도는 $\mathcal{O}(N \lg N)$ 입니다.